



Plan de mejoramiento de 15 días

Desarrollo web full stack: del color al reporte

Instructor responsable: Ing. José Andrés Zambrano Díaz

Versión 1.0 · Modalidad presencial/autónoma · Intensidad total: 60 horas (4 h/día)

Entrega final: lunes 28 de septiembre de 2026, 5:00 p. m. – Microsoft Teams

1. Identificación del aprendiz

Este formato se diligencia antes de iniciar el plan. El aprendiz y el instructor firman al final del documento como constancia de acuerdo.

Nombre del aprendiz		Documento	
Número de ficha		Trimestre	
Fecha de inicio		Fecha de entrega	Lunes 28 de septiembre de 2026 · 5:00 p. m.
Medio de entrega	Microsoft Teams — tarea asignada al aprendiz en el equipo del programa. No se reciben entregas por correo, memoria USB ni mensajería instantánea.		
Competencia	Desarrollar el sistema de información que cumpla con los requisitos de la solución informática.		
Resultados de aprendizaje por recuperar	Construir interfaces de usuario aplicando principios de diseño visual y accesibilidad. Codificar módulos del sistema con control de acceso, persistencia y generación de reportes.		
Causal del plan			

Fecha y hora límite: lunes 28 de septiembre de 2026, 5:00 p. m.

La entrega se realiza **únicamente a través de Microsoft Teams**, en la tarea creada para este plan. La plataforma cierra automáticamente a la hora indicada y registra la marca de tiempo de cada envío; un envío posterior queda señalado como tardío y se califica como tal.

Qué se adjunta en la tarea de Teams:

- Enlace público al repositorio de GitHub (no un archivo comprimido del proyecto).
- Archivo comprimido con la aplicación y los tres scripts SQL, como respaldo.
- Los tres reportes exportados en PDF.
- Bitácora de los quince días en un solo PDF.
- Video corto de recorrido de la aplicación, o el enlace a este.

La sustentación se agenda dentro de los tres días hábiles siguientes a la entrega. Sin entrega en Teams no hay sustentación, y sin sustentación el plan no se aprueba.

2. Propósito y alcance

El aprendiz no aprueba este plan entregando ejercicios sueltos. Entrega **una sola aplicación web funcional**, construida día a día, que al final del día 15 debe iniciar sesión de forma segura, mostrar un tablero con menú lateral, administrar datos mediante CRUD, graficar indicadores y emitir reportes con la identidad visual del software.

Producto integrador: ZD.TechLab — Panel de gestión de tienda tecnológica.

Aplicación web para administrar el inventario y las ventas de una tienda de accesorios y equipos de cómputo. Módulos mínimos: `usuarios` , `categorias` , `productos` , `clientes` , `pedidos` y `detalle_pedido` .

El aprendiz define el nombre comercial, el logo y la paleta de su propia versión del software en el día 2, y esa identidad se mantiene en el tablero y en todos los reportes.

Reglas de entrega

- Todo el material se sube a la tarea de **Microsoft Teams** antes del **lunes 28 de septiembre de 2026 a las 5:00 p. m.**
- Un repositorio en GitHub creado el día 1, con **mínimo un commit por día** y mensajes descriptivos. Un repositorio con todo subido el último día se califica en cero.
- Bitácora diaria escrita a mano o en documento propio: qué hizo, qué error encontró, cómo lo resolvió. Se anexa fotografiada o escaneada.
- Todo código entregado debe poder ser explicado línea por línea en la sustentación del día 15.

- Cada día tiene una evidencia concreta. Sin la evidencia del día anterior no se avanza al siguiente.

Validación de autoría. Durante la sustentación el instructor solicitará al aprendiz, en vivo y sin apoyo externo: (a) explicar el flujo de una consulta desde el formulario hasta la base de datos; (b) modificar un campo del CRUD y volver a ejecutarlo; (c) cambiar el tipo de gráfico de Chart.js; (d) justificar la paleta de color elegida usando los conceptos del día 1. El uso de asistentes de IA como apoyo de estudio está permitido; entregar código que el aprendiz no comprende, no.

3. Cronograma general

Fase	Días	Eje temático	Entregable de fase
I	1 – 6	Teoría del color, HTML5 y CSS3 desde cero, maquetación responsive	Prototipo visual navegable del sistema
II	7 – 8	JavaScript desde cero, DOM, eventos y validación	Formularios validados en cliente
III	9 – 11	PHP, conexión PDO, inicio de sesión seguro y manejo de sesiones	Acceso protegido con roles
IV	12 – 15	Dashboard con menú lateral, CRUD, vistas SQL, Chart.js y reportes	Aplicación completa sustentada

Índice de sesiones

- 01 Teoría del color aplicada al software
- 02 Paleta de marca, contraste y accesibilidad
- 03 HTML5 semántico desde cero
- 04 CSS desde cero: selectores y modelo de caja
- 05 Maquetación con Flexbox y CSS Grid
- 06 Responsive y componentes de interfaz
- 07 JavaScript desde cero
- 08 DOM, eventos y validación de formularios

- 09 PHP y conexión PDO a MySQL
- 10 Inicio de sesión seguro
- 11 Manejo de sesiones y control de acceso
- 12 Dashboard con menú lateral izquierdo
- 13 CRUD completo sobre la base de datos
- 14 Vistas SQL y gráficos con Chart.js
- 15 Reportes con logo y sustentación final

1 DÍA

Teoría del color aplicada al software

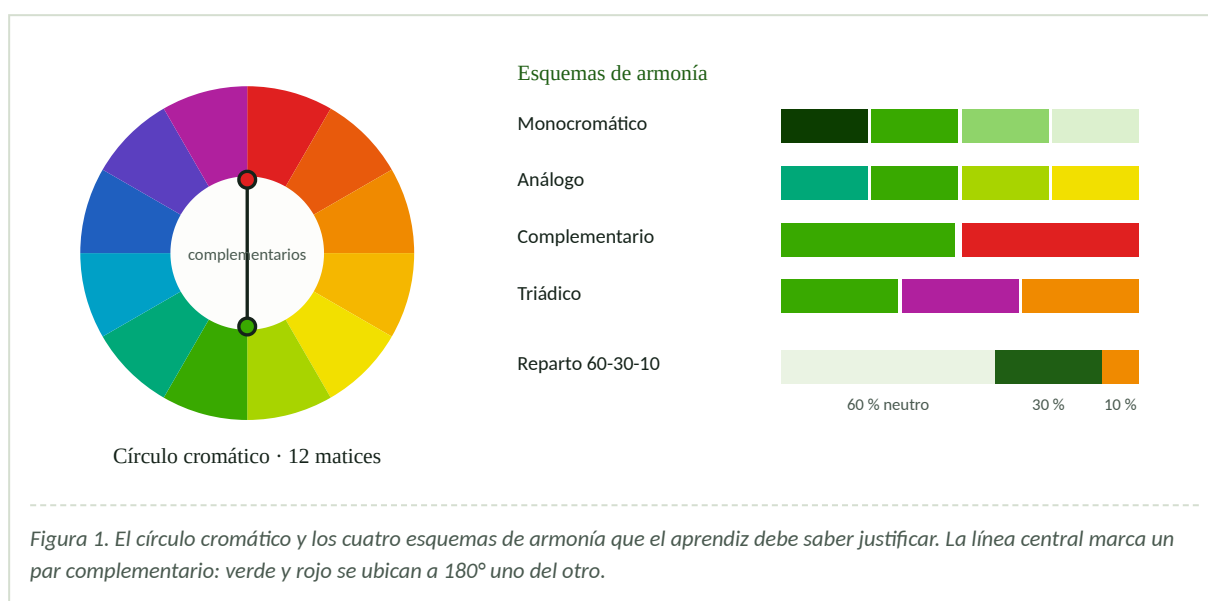
4 horas · Fundamentación · Sin código todavía

Objetivo

Explicar por qué un color funciona o no dentro de una interfaz, usando el círculo cromático, las propiedades del color y los esquemas de armonía, en lugar de elegir colores por gusto.

Contenidos

- Colores primarios, secundarios y terciarios. Diferencia entre síntesis aditiva (RGB, pantalla) y sustractiva (CMYK, impresión). Todo lo que se programa vive en RGB.
- Las tres propiedades: **matiz** (qué color es), **saturación** (qué tan puro) y **luminosidad** (qué tan claro). El modelo `hsl()` de CSS las expone directamente y por eso es más fácil de razonar que `#hex`.
- Esquemas de armonía: monocromático, análogo, complementario, complementario dividido y triádico. Cuándo conviene cada uno en una aplicación de gestión.
- Temperatura y significado: colores cálidos avanzan y llaman la atención (alertas, botones de acción); fríos retroceden y organizan (fondos, barras de navegación).
- Regla 60-30-10 para repartir el color en una pantalla: 60 % neutro dominante, 30 % color de apoyo, 10 % acento.



Ejemplo guía

El mismo matiz expresado en los tres sistemas que se usan en CSS. Conviene trabajar en

`hsl()` porque aclarar u oscurecer es cambiar un solo número:

```
/* El verde institucional, escrito de tres formas equivalentes */
--verde: #39A900;
--verde: rgb(57, 169, 0);
--verde: hsl(100, 100%, 33%);

/* Una escala completa cambiando solo la luminosidad */
--verde-900: hsl(100, 100%, 12%);
--verde-700: hsl(100, 100%, 22%);
--verde-500: hsl(100, 100%, 33%); /* base */
--verde-300: hsl(100, 60%, 62%);
--verde-100: hsl(100, 45%, 90%);
```

Actividad

1. Dibujar a mano el círculo cromático de 12 matices y rotularlo. Va en la bitácora.
2. Tomar tres aplicaciones reales de gestión (por ejemplo un banco, un ERP y una tienda en línea), capturar sus pantallas e identificar en cada una: color dominante, color de apoyo, color de acento y esquema de armonía utilizado.
3. Construir una escala de cinco tonos a partir de un solo matiz, usando `hsl()`, y anotar el valor hexadecimal equivalente de cada uno.

Evidencia del día

Documento `dia01-color.pdf` con el círculo dibujado, las tres capturas analizadas y la escala de cinco tonos. Repositorio de GitHub creado con `README.md` y primer commit.

Criterio de evaluación

Cumple si nombra correctamente el esquema de armonía de cada captura y explica por qué el color de acento es el que es. No cumple si describe colores solo como "bonito" o "serio" sin referencia al círculo cromático.

2^{DÍA}

Paleta de marca, contraste y accesibilidad

4 horas · Fundamentación + decisión de diseño

Objetivo

Convertir la teoría del día anterior en la paleta definitiva del software, verificada con criterios de contraste, y dejarla escrita como variables CSS reutilizables durante los quince días.

Contenidos

- Contraste de luminancia y norma WCAG 2.1: relación mínima **4.5:1** para texto normal y **3:1** para texto grande o elementos de interfaz. Nivel AA como piso obligatorio.
- Por qué el color nunca puede ser el único portador de información: un estado "activo/inactivo" necesita también texto o ícono, porque cerca del 8 % de los hombres tiene alguna deficiencia en la visión del color.
- Colores semánticos de sistema: éxito, advertencia, error e información. Son convenciones, no decoración.
- Tokens de diseño: nombrar el color por su función (`--color-accion`) y no por su apariencia (`--verde`), para poder cambiar la marca sin reescribir la hoja de estilos.
- Herramientas de verificación: WebAIM Contrast Checker, Coolers, Adobe Color, y el panel de accesibilidad de las DevTools del navegador.

Verificación de contraste (WCAG 2.1)

Guardar cambios ✕ 3.1:1 – falla AA	Guardar cambios ✓ 8.9:1 – cumple AAA	Guardar cambios ✕ 1.6:1 – ilegible	Guardar cambios 13.4:1 – cumple AAA
--	--	--	---

Colores semánticos del sistema

Éxito · registro guardado #2E7D32	Advertencia · stock bajo #A85E00	Error · credenciales inválidas #B3261E	Información · sesión activa #15506B
---	--	--	---

Cada estado combina color + ícono + texto. Quitando el color, el mensaje sigue siendo comprensible.

Figura 2. La misma etiqueta sobre cuatro fondos. El verde puro sobre blanco es la trampa más común: se ve bien pero no alcanza el mínimo legal de contraste para texto.

Ejemplo guía

Archivo `css/tokens.css`. Se crea hoy y se importa en todas las páginas del proyecto hasta el día 15:

```
:root{
  /* Neutros — el 60 % de la pantalla */
  --superficie:    #F7F8F6;
  --superficie-alta: #FFFFFF;
  --borde:         #DDE3DA;

  /* Marca — el 30 % */
  --color-marca:   #1F5E14;
  --color-marca-sua: #EAF3E3;

  /* Acción y acento — el 10 % */
  --color-accion:  #39A900;
  --color-acento:  #F08A00;

  /* Texto */
  --texto:         #15231A; /* 13.4:1 sobre --superficie */
  --texto-tenue:   #4A5A50; /* 6.2:1 sobre --superficie */

  /* Semánticos */
  --exito: #2E7D32; --advertencia: #A85E00;
  --error: #B3261E; --info: #15506B;
}
```

Actividad

1. Definir el nombre comercial y el logo del software. El logo puede hacerse en Figma, Inkscape o a mano y digitalizarse; debe exportarse en `PNG` y en `SVG` y guardarse en `/assets/img/logo.svg`. Este logo aparecerá después en el tablero y en todos los reportes.
2. Construir la paleta completa en `css/tokens.css` siguiendo la estructura del ejemplo.
3. Verificar en WebAIM cada combinación texto/fondo que se piensa usar y registrar la relación de contraste obtenida en una tabla dentro de la bitácora. Ninguna combinación de texto puede quedar por debajo de 4.5:1.

Evidencia del día

`css/tokens.css` y `assets/img/logo.svg` en el repositorio. Tabla de contrastes con capturas de la herramienta de verificación. Commit del día 2.

Criterio de evaluación

Cumple si la paleta usa nombres por función, respeta el esquema de armonía declarado el día 1 y todas las combinaciones alcanzan AA. No cumple si los tokens se llaman `--azul1`, `--azul2` o si hay contrastes por debajo del mínimo.

3^{DÍA}

HTML5 semántico desde cero

4 horas · Codificación

Objetivo

Escribir el esqueleto de las páginas del sistema usando etiquetas que describen el significado del contenido, no su apariencia.

Contenidos

- Anatomía de un documento: `<!DOCTYPE html>` , `<html lang="es">` , `<head>` con `charset` y `viewport` , y `<body>` .
- Etiquetas de estructura: `header` , `nav` , `main` , `aside` , `section` , `article` , `footer` . Cuándo sí corresponde un `div` (agrupación puramente visual, sin significado).
- Jerarquía de encabezados: un solo `h1` por página y sin saltar niveles. Es lo que usan los lectores de pantalla y los buscadores para entender la página.
- Listas, tablas de datos con `thead` , `tbody` , `th scope` y `caption` .
- Formularios: `label for` asociado a cada campo, tipos de `input` (`email` , `password` , `number` , `date`), `required` , `fieldset` y `legend` .
- Atributos de accesibilidad básicos: `alt` en imágenes, `aria-label` en botones de solo ícono.

Sin semántica

```
div class="top"
```

```
div class="menu"
```

```
div id="izq"
```

```
div id="cont"
```

```
div class="abajo"
```

El navegador no sabe qué es cada bloque.

Con semántica

```
<header>
```

```
<nav>
```

```
<aside>
```

```
<main>
```

```
<section>
```

```
<article>
```

```
<footer>
```

Cada bloque comunica su función al navegador y al lector de pantalla.

Figura 3. La misma pantalla maquetada de dos formas. A la derecha, un lector de pantalla puede ofrecer "saltar a contenido principal"; a la izquierda no.

Ejemplo guía

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Ingreso – ZD.TechLab</title>
  <link rel="stylesheet" href="css/tokens.css">
</head>
<body>
  <main class="pantalla-ingreso">
    
    <h1>Ingreso al panel de gestión</h1>

    <form action="login.php" method="post" novalidate>
      <fieldset>
        <legend>Credenciales</legend>

        <label for="correo">Correo electrónico</label>
        <input type="email" id="correo" name="correo" required autocomplete="username">

        <label for="clave">Contraseña</label>
        <input type="password" id="clave" name="clave" required
          autocomplete="current-password" minlength="8">
      </fieldset>
      <button type="submit">Iniciar sesión</button>
    </form>
  </main>
</body>
</html>
```

Actividad

1. Crear la estructura de carpetas del proyecto: `/assets/img` , `/css` , `/js` , `/app` , `/sql` .
2. Escribir sin estilos tres páginas: `login.html` , `dashboard.html` y `productos.html` . Solo HTML semántico.
3. En `productos.html` incluir una tabla de datos correctamente construida y un formulario de registro de producto con todos sus `label` asociados.
4. Validar las tres páginas en validator.w3.org hasta obtener cero errores y adjuntar la captura.

Evidencia del día

Tres archivos HTML en el repositorio y captura del validador W3C sin errores para cada uno.

Criterio de evaluación

Cumple si las páginas validan sin errores, usan al menos cinco etiquetas semánticas distintas y cada campo de formulario tiene su `label` . No cumple si la maquetación se construyó únicamente con `div` y `span` .

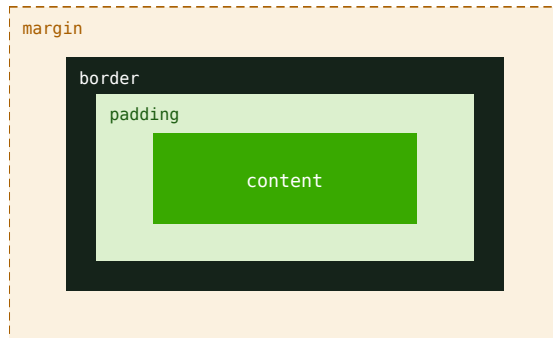
Objetivo

Dar apariencia a las páginas del día anterior aplicando la paleta del día 2, y entender por qué un estilo se sobrescribe a otro.

Contenidos

- Formas de vincular CSS y por qué el estilo en línea es la peor de las tres.
- Selectores: elemento, clase, identificador, descendiente, hijo directo, agrupado, atributo y pseudoclases (`:hover` , `:focus-visible` , `:nth-child`).
- **Especificidad** y cascada: por qué `#menu a` gana sobre `.enlace` . El costo real de usar `!important` .
- Modelo de caja: contenido, `padding` , `border` , `margin` . La diferencia entre `box-sizing: content-box` y `border-box` , y por qué hoy se normaliza siempre a `border-box` .
- Unidades: `px` , `%` , `rem` , `em` , `vh/vw` , `ch` . Escala tipográfica y longitud de línea cómoda (entre 45 y 75 caracteres).
- Variables CSS: cómo consumir los tokens del día 2 con `var(--color-accion)` .

Modelo de caja



```
*, ::before, ::after { box-sizing: border-box; }
```

Con `border-box`, `width` incluye `padding` y `borde`: el elemento mide lo que dice medir.

Especificidad: quién gana

<code>p</code>	0-0-1
<code>.tarjeta p</code>	0-1-1
<code>.panel .tarjeta p</code>	0-2-1
<code>#menu .tarjeta p</code>	1-1-1
<code>color: red !important</code>	rompe todo

Se lee como (identificadores) - (clases) - (elementos).

Gana el de mayor valor; a igual valor, gana el último escrito.

Si necesita `!important` para que funcione, el problema está en la estructura.

Figura 4. Izquierda: las cuatro capas del modelo de caja. Derecha: cómo el navegador resuelve dos reglas que afectan al mismo elemento.

Ejemplo guía

```
@import url('tokens.css');

*, *::before, *::after { box-sizing: border-box; }

body{
  margin:0;
  font-family: system-ui, "Segoe UI", Roboto, sans-serif;
  font-size:16px;
  line-height:1.6;
  color: var(--texto);
  background: var(--superficie);
}

h1{ font-size:1.9rem; line-height:1.2; margin:0 0 .5rem; }
p { max-width:70ch; }

.boton{
  background: var(--color-accion);
  color:#fff;
  border:none;
  padding:.7rem 1.4rem;
  font-size:1rem;
  cursor:pointer;
  transition: background .18s ease;
}
.boton:hover{ background: var(--color-marca); }
.boton:focus-visible{ outline:3px solid var(--color-acento); outline-offset:2px; }
```

La regla `:focus-visible` no es opcional: es lo que permite usar el sistema con el teclado.

Actividad

1. Crear `css/estilos.css` que importe los tokens y dar estilo completo a `login.html`.
2. Construir una escala tipográfica de cinco tamaños en `rem` y aplicarla.
3. Estilizar la tabla de productos: encabezado con color de marca, filas alternas y estado de `:hover`.
4. Ejercicio de diagnóstico: dado un fragmento con tres reglas en conflicto entregado por el instructor, predecir por escrito qué color se aplicará y comprobarlo en el navegador.

Evidencia del día

`css/estilos.css`, pantalla de ingreso estilizada, captura antes/después y la predicción escrita del ejercicio de especificidad.

Criterio de evaluación

Cumple si todos los colores provienen de variables, existe estado visible de foco y no aparece ningún `!important`. No cumple si hay valores hexadecimales sueltos repartidos por la hoja.

Objetivo

Armar el esqueleto visual del tablero — barra superior, menú lateral izquierdo y área de contenido — usando Grid para la estructura general y Flexbox para el interior de cada componente.

Contenidos

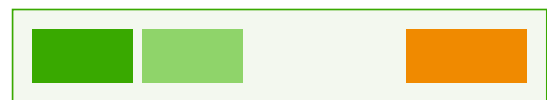
- Flexbox: eje principal y eje transversal, `flex-direction`, `justify-content`, `align-items`, `gap`, `flex-wrap` y la abreviatura `flex: 1 1 auto`.
- CSS Grid: `grid-template-columns`, `grid-template-rows`, la unidad `fr`, `minmax()`, `repeat()` y `auto-fit`.
- `grid-template-areas`: nombrar las zonas del tablero y dibujar la maqueta en la propia hoja de estilos.
- Criterio de elección: Grid cuando el reparto es bidimensional (la estructura de la página); Flexbox cuando es una sola línea o columna (una barra de botones, el contenido de una tarjeta).
- `position: sticky` para mantener fijo el encabezado de la tabla y la barra superior.

Grid: estructura de la página



```
grid-template-columns: 240px 1fr;
```

Flexbox: interior de un componente



`justify-content: space-between`



`justify-content: center`



`flex: 1 1 auto` (los hijos se reparten el espacio)

Figura 5. Izquierda: las cuatro zonas nombradas del tablero. Derecha: tres comportamientos de Flexbox que el aprendiz debe reconocer a simple vista.

Ejemplo guía

```
.panel{
  display:grid;
  grid-template-columns: 240px 1fr;
  grid-template-rows: 60px 1fr auto;
  grid-template-areas:
    "barra barra"
    "menu contenido"
    "pie pie";
  min-height:100vh;
}
.panel__barra { grid-area: barra; display:flex;
  align-items:center; justify-content:space-between;
  padding:0 1.2rem; background:var(--color-marca); color:#fff; }
.panel__menu { grid-area: menu; background:#15231A; }
.panel__contenido { grid-area: contenido; padding:1.6rem; overflow-y:auto; }
.panel__pie { grid-area: pie; padding:.8rem 1.2rem;
  border-top:1px solid var(--borde); font-size:.85rem; }

/* Tarjetas de indicadores que se reacomodan solas */
.indicadores{
  display:grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap:1rem;
}
```

Actividad

1. Maquetar `dashboard.html` completo con Grid: barra superior, menú lateral izquierdo de 240 px, contenido y pie.
2. Crear la fila de cuatro tarjetas de indicadores con `auto-fit` y verificar que se reacomodan al angostar la ventana.
3. Fijar la barra superior con `position: sticky` y comprobar el desplazamiento con una tabla larga.
4. Escribir en la bitácora, con sus palabras, en qué caso usó Grid y en cuál Flexbox, y por qué.

Evidencia del día

`dashboard.html` maquetado (aún sin datos reales) y tres capturas a distintos anchos de ventana.

Criterio de evaluación

Cumple si la estructura usa `grid-template-areas`, el menú lateral conserva su ancho y el contenido ocupa el resto sin desbordarse. No cumple si la maquetación se logró con `float` o con posicionamiento absoluto.

6^{DÍA}

Diseño responsive y componentes de interfaz

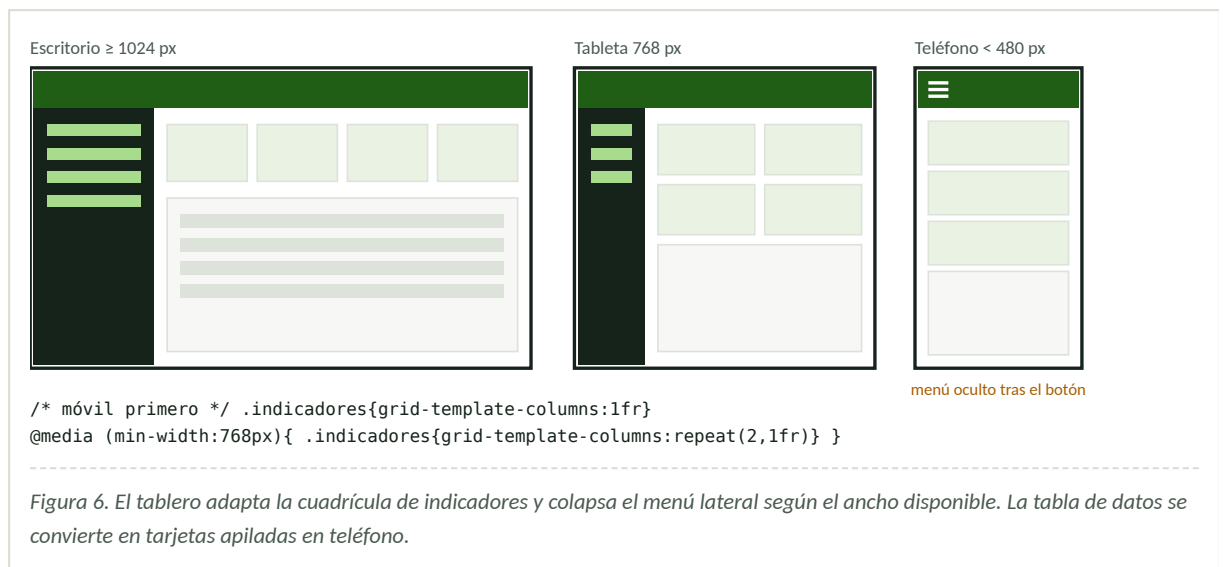
4 horas · Codificación · Cierre de fase I

Objetivo

Lograr que el prototipo funcione en teléfono, tableta y escritorio, y consolidar los componentes reutilizables que se usarán durante el resto del plan.

Contenidos

- Enfoque *mobile first*: escribir primero el estilo de pantalla pequeña y ampliar con `min-width`.
- *Media queries* y puntos de quiebre razonables: 480 px, 768 px, 1024 px, 1280 px. El quiebre se define donde el diseño se rompe, no por el modelo de teléfono.
- Imágenes flexibles, `object-fit` y unidades relativas.
- El patrón del menú lateral colapsable: visible en escritorio, oculto tras un botón en móvil.
- Biblioteca de componentes: botones (primario, secundario, peligro), campos de formulario con estado de error, tarjeta de indicador, tabla de datos, mensaje de alerta y ventana modal.
- Estados obligatorios de todo componente interactivo: normal, `:hover`, `:focus-visible`, activo y deshabilitado.



Ejemplo guía

```
/* Base: teléfono */
.panel{ grid-template-columns: 1fr;
        grid-template-areas: "barra" "contenido" "pie"; }
.panel__menu{ position:fixed; inset:0 auto 0 0; width:240px;
              transform:translateX(-100%); transition:transform .25s ease; }
.panel__menu.abierto{ transform:translateX(0); }
.boton-menu{ display:inline-flex; }

/* Escritorio */
@media (min-width: 1024px){
  .panel{ grid-template-columns:240px 1fr;
          grid-template-areas:"barra barra" "menu contenido" "pie pie"; }
  .panel__menu{ position:static; transform:none; }
  .boton-menu{ display:none; }
}

@media (prefers-reduced-motion: reduce){
  *{ transition-duration:.01ms !important; }
}
```

Actividad

1. Aplicar *mobile first* a todo el prototipo y definir los puntos de quiebre.
2. Implementar el menú lateral colapsable (por ahora, el botón se conecta el día 8 con JavaScript; hoy se resuelve solo con CSS y la clase `.abierto` aplicada a mano).
3. Crear `componentes.html` : una página que muestre todos los componentes con sus cinco estados. Es la guía de estilo del proyecto.
4. Probar en las herramientas de desarrollo del navegador a 360, 768 y 1440 px, y en un teléfono real.

Evidencia del día

`componentes.html` publicado, capturas en los tres anchos y video corto (máximo 60 segundos) recorriendo el prototipo en el teléfono.

Criterio de evaluación

Cumple si no aparece desplazamiento horizontal en ningún ancho, el menú colapsa correctamente y todos los componentes tienen los cinco estados documentados. No cumple si el diseño se escribió para escritorio y luego se "arregló" con anchos fijos.

7 DÍA**JavaScript desde cero**

4 horas · Codificación

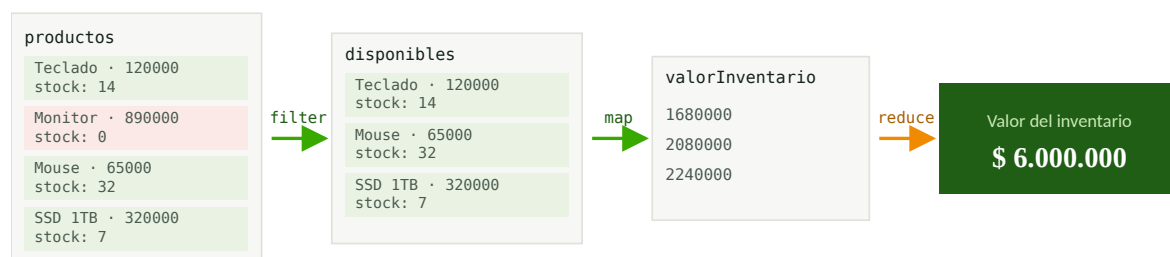
Objetivo

Dominar la sintaxis base del lenguaje y las estructuras de datos que después alimentarán las tablas y los gráficos del tablero.

Contenidos

- Declaración de variables: `let` y `const` ; por qué `var` quedó atrás. Ámbito de bloque.
- Tipos primitivos y el peligro de la comparación débil: `==` frente a `===` .
- Estructuras de control: `if / else if / else` , `switch` , operador ternario, `for` , `for...of` , `while` .
- Funciones declaradas, expresadas y flecha. Parámetros por defecto y retorno.
- Arreglos y sus métodos de trabajo: `push` , `filter` , `map` , `find` , `reduce` , `sort` . Son exactamente los que se usarán para preparar los datos de Chart.js.
- Objetos, notación de punto y corchete, desestructuración, y arreglos de objetos como representación de una tabla de base de datos.
- `JSON.stringify` y `JSON.parse` : el formato en el que PHP entregará los datos al navegador.
- Depuración con `console.log` , `console.table` y puntos de interrupción en el navegador.

De los datos crudos al dato del gráfico



```
const total = productos.filter(p => p.stock > 0).map(p => p.precio * p.stock).reduce((a,b) => a+b, 0);
```

Figura 7. La cadena `filter` → `map` → `reduce` es la misma que el día 14 producirá los números de las tarjetas de indicadores del tablero.

Ejemplo guía

```
const productos = [
  { id:1, nombre:"Teclado mecánico", categoria:"Periféricos", precio:120000, stock:14 },
  { id:2, nombre:"Monitor 24\"", categoria:"Pantallas", precio:890000, stock:0 },
  { id:3, nombre:"Mouse inalámbrico", categoria:"Periféricos", precio:65000, stock:32 },
  { id:4, nombre:"SSD 1 TB", categoria:"Almacenamiento", precio:320000, stock:7 }
];

const disponibles = productos.filter(p => p.stock > 0);

const valorInventario = disponibles
  .map(p => p.precio * p.stock)
  .reduce((suma, v) => suma + v, 0);

const porCategoria = productos.reduce((acc, p) => {
  acc[p.categoria] = (acc[p.categoria] ?? 0) + p.stock;
  return acc;
}, {});

const moneda = new Intl.NumberFormat("es-CO",
  { style:"currency", currency:"COP", maximumFractionDigits:0 });

console.table(disponibles);
console.log(moneda.format(valorInventario)); // $ 6.000.000
console.log(porCategoria); // { Periféricos: 46, Pantallas: 0, Almacenamiento: 7 }
```

Actividad

1. Crear `js/datos-prueba.js` con un arreglo de al menos quince productos y ocho pedidos, con la misma forma que tendrán después las tablas de la base de datos.
2. Resolver con métodos de arreglo, sin usar `for`: producto más caro, total de unidades por categoría, listado de productos con stock menor a cinco, y promedio de precio.
3. Formatear todos los valores monetarios con `Intl.NumberFormat` en pesos colombianos.
4. Escribir en la bitácora la diferencia entre `map` y `forEach` con un ejemplo propio.

Evidencia del día

`js/datos-prueba.js` y `js/ejercicios.js` con los cuatro cálculos resueltos, más captura de la consola mostrando los resultados.

Criterio de evaluación

Cumple si los cuatro cálculos se resuelven con métodos de arreglo encadenados y el aprendiz explica qué recibe y qué devuelve cada uno. No cumple si se usan ciclos `for` anidados para lo que un `reduce` resuelve.

8^{DÍA}

DOM, eventos y validación de formularios

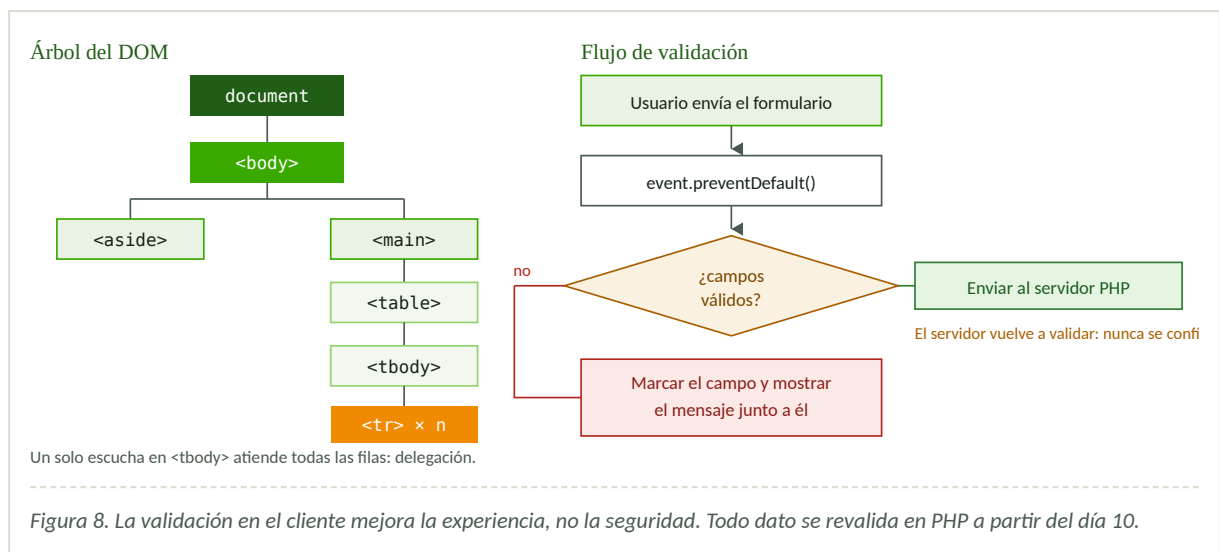
4 horas · Codificación · Cierre de fase II

Objetivo

Hacer que la interfaz responda: pintar datos en la tabla, abrir y cerrar el menú lateral, y validar los formularios antes de enviarlos al servidor.

Contenidos

- El DOM como árbol. `querySelector` y `querySelectorAll`; por qué se prefieren a `getElementById`.
- Lectura y escritura: `textContent` frente a `innerHTML`, y el riesgo de XSS al inyectar contenido del usuario.
- Crear elementos con `createElement`, `append` y fragmentos; manipular clases con `classList`.
- Eventos: `addEventListener`, objeto `event`, `preventDefault()`, propagación y **delegación de eventos** (un solo escucha en la tabla en lugar de uno por botón).
- Atributos `data-*` y `dataset` para asociar el identificador de un registro a su botón de editar o eliminar. Se evita el `onclick` en línea.
- Validación: API de restricciones de HTML5 (`checkValidity`, `setCustomValidity`), expresiones regulares y mensajes de error accesibles con `aria-describedby`.



Ejemplo guía

```
const tbody = document.querySelector("#tabla-productos tbody");
const plantilla = p => {
  const tr = document.createElement("tr");
  tr.innerHTML = `
    <td>${p.nombre}</td>
    <td>${moneda.format(p.precio)}</td>
    <td>${p.stock}</td>
    <td>
      <button class="boton-mini" data-accion="editar" data-id="${p.id}">Editar</button>
      <button class="boton-mini boton-peligro" data-accion="eliminar" data-
id="${p.id}">Eliminar</button>
    </td>`;
  return tr;
};

function pintar(lista){
  tbody.replaceChildren(...lista.map(plantilla));
}

/* Un solo escucha para toda la tabla: delegación de eventos */
tbody.addEventListener("click", (e) => {
  const boton = e.target.closest("button[data-accion]");
  if (!boton) return;
  const { accion, id } = boton.dataset;
  if (accion === "editar") abrirFormulario(Number(id));
  if (accion === "eliminar") confirmarBorrado(Number(id));
});

/* Menú lateral */
document.querySelector(".boton-menu")
  .addEventListener("click", () =>
    document.querySelector(".panel__menu").classList.toggle("abierto"));

/* Validación */
const form = document.querySelector("#form-producto");
form.addEventListener("submit", (e) => {
  const precio = form.elements.precio;
  precio.setCustomValidity(
    Number(precio.value) <= 0 ? "El precio debe ser mayor que cero." : ""
  );
  if (!form.checkValidity()){
    e.preventDefault();
    form.reportValidity();
  }
});
```

Actividad

1. Pintar la tabla de productos desde `datos-prueba.js`, sin escribir ninguna fila a mano en el HTML.
2. Implementar buscador en vivo sobre la tabla con `input` y `filter`.
3. Conectar el botón del menú lateral y verificar que también responde con la tecla `Enter`.
4. Validar el formulario de producto: nombre obligatorio de mínimo tres caracteres, precio mayor que cero, stock entero no negativo y categoría seleccionada. Los mensajes deben aparecer junto al campo, no en un `alert`.

Evidencia del día

Prototipo interactivo funcionando y video de 60 segundos mostrando: filtrado en vivo, apertura del menú y los cuatro mensajes de validación disparándose.

Criterio de evaluación

Cumple si no existe ningún `onclick` en el HTML, la tabla usa delegación de eventos y los errores se muestran junto al campo correspondiente. No cumple si la validación se resuelve con ventanas `alert`.

9 DÍA**PHP desde cero y conexión PDO a MySQL**

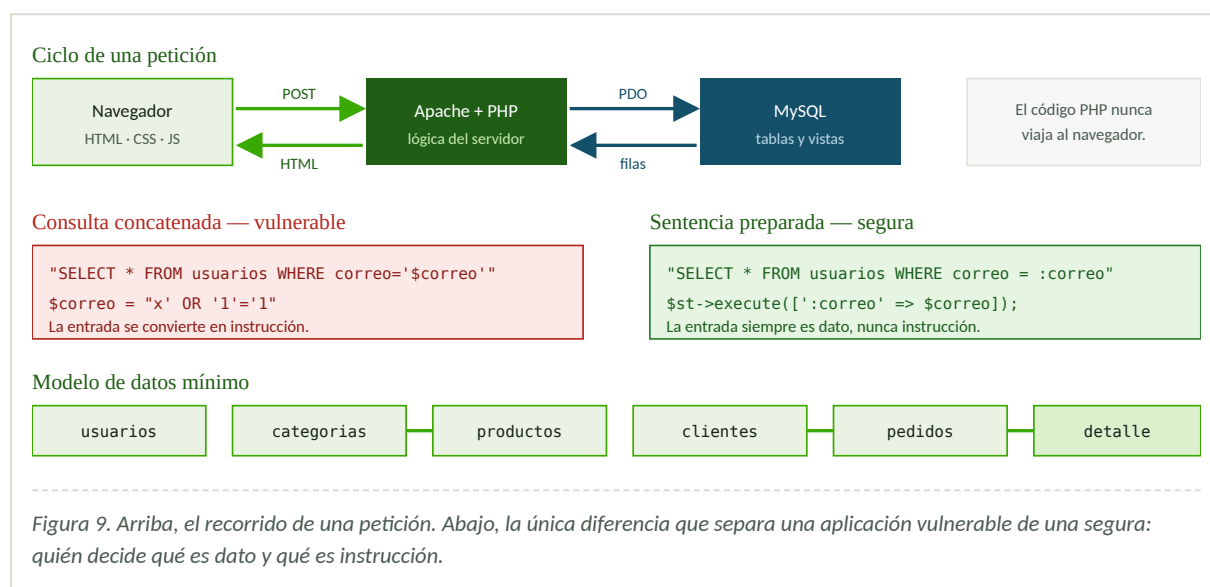
4 horas · Codificación

Objetivo

Montar el entorno de servidor, crear la base de datos del proyecto y establecer una conexión reutilizable con PDO usando sentencias preparadas desde la primera consulta.

Contenidos

- Cómo funciona una petición: el navegador pide, Apache entrega el archivo a PHP, PHP consulta MySQL y devuelve HTML ya resuelto. El usuario nunca ve el código PHP.
- Sintaxis base: variables, tipos, arreglos asociativos, estructuras de control, funciones, `require_once` e inclusión de plantillas.
- `$_GET`, `$_POST` y por qué todo lo que llega por ahí es hostil hasta demostrar lo contrario.
- PDO frente a `mysqli`: portabilidad, manejo de errores por excepciones y sentencias preparadas.
- **Sentencias preparadas con marcadores nombrados**: cómo evitan la inyección SQL separando la instrucción de los datos.
- Configuración obligatoria del objeto PDO: modo de error por excepción, modo de obtención asociativo y emulación de preparadas desactivada.
- Credenciales fuera del código versionado: archivo de configuración excluido en `.gitignore`.
- Diseño de la base de datos del proyecto y carga de datos de prueba.



Ejemplo guía

Archivo `app/config/conexion.php` — se escribe una vez y se reutiliza en todo el proyecto:

```
<?php
declare(strict_types=1);

final class Conexion
{
    private static ?PDO $pdo = null;

    public static function obtener(): PDO
    {
        if (self::$pdo === null) {
            $cfg = require __DIR__ . '/credenciales.php'; // excluido del repositorio
            $dsn = "mysql:host={$cfg['host']};dbname={$cfg['bd']};charset=utf8mb4";

            self::$pdo = new PDO($dsn, $cfg['usuario'], $cfg['clave'], [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false,
            ]);
        }
        return self::$pdo;
    }
}
```

Primera consulta preparada, en `app/modelos/ProductoModelo.php`:

```
<?php
function buscarProductos(PDO $pdo, string $texto, int $limite = 20): array
{
    $sql = "SELECT p.id, p.nombre, p.precio, p.stock, c.nombre AS categoria
            FROM productos p
            INNER JOIN categorias c ON c.id = p.categoria_id
            WHERE p.nombre LIKE :texto
            ORDER BY p.nombre
            LIMIT :limite";

    $st = $pdo->prepare($sql);
    $st->bindValue(':texto', '%' . $texto . '%', PDO::PARAM_STR);
    $st->bindValue(':limite', $limite, PDO::PARAM_INT);
    $st->execute();

    return $st->fetchAll();
}
```

Actividad

1. Instalar XAMPP o Laragon y comprobar la versión de PHP (mínimo 8.0) con `phpinfo()`.
2. Escribir `sql/estructura.sql` con las seis tablas, claves foráneas, tipos adecuados e índices. Escribir `sql/datos.sql` con mínimo veinte productos, diez clientes y quince pedidos.
3. Implementar `conexion.php` con el patrón del ejemplo y agregar `credenciales.php` al `.gitignore`, dejando en su lugar un `credenciales.example.php`.
4. Reemplazar `datos-prueba.js`: la tabla de productos ahora se llena desde MySQL.
5. Demostración obligatoria: intentar la inyección `' OR '1'='1` en el buscador y documentar que no ocurre nada.

Evidencia del día

Scripts SQL en el repositorio, `conexion.php`, listado de productos leído desde la base de datos y captura del intento de inyección fallido.

Criterio de evaluación

Cumple si toda consulta usa `prepare()` con marcadores, el archivo de credenciales no está en el repositorio y las claves foráneas están declaradas. No cumple si aparece una sola consulta con variables concatenadas.

10^{DÍA}

Inicio de sesión seguro

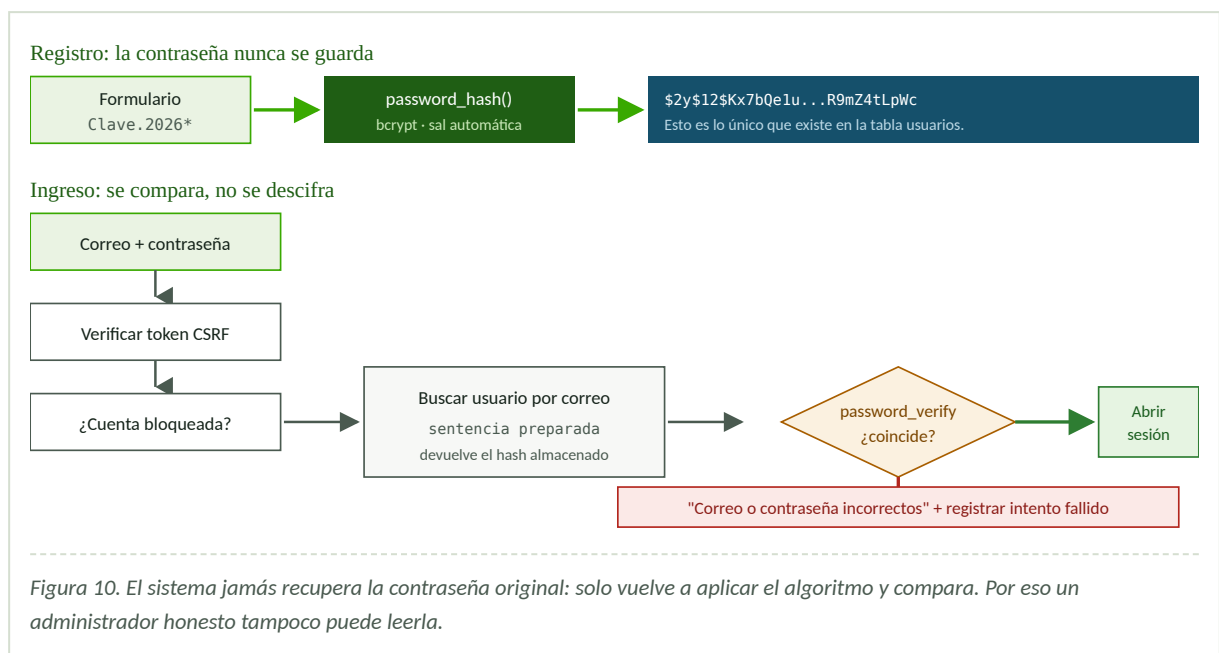
4 horas · Codificación · Núcleo del plan

Objetivo

Implementar el registro y el ingreso de usuarios almacenando contraseñas con hash moderno, validando en el servidor y protegiendo el formulario contra los ataques más frecuentes.

Contenidos

- Por qué una contraseña **nunca** se guarda en texto plano ni cifrada de forma reversible ni con MD5 o SHA-1. Diferencia entre cifrar y aplicar hash.
- `password_hash()` con `PASSWORD_DEFAULT` : la sal se genera sola y va dentro del resultado. `password_verify()` compara en tiempo constante.
`password_needs_rehash()` para migrar el costo con el tiempo.
- Mensajes de error genéricos: informar "credenciales incorrectas" y no "el correo no existe", para no permitir enumerar cuentas.
- Protección CSRF: token aleatorio con `random_bytes()` guardado en sesión, incrustado en el formulario y comparado con `hash_equals()` .
- Limitación de intentos: bloqueo temporal de la cuenta tras cinco fallos, registrados en tabla `intentos_acceso` .
- Escapar la salida con `htmlspecialchars()` para evitar XSS al mostrar datos del usuario.
- Validación y saneamiento en servidor con `filter_var` y verificación de longitud mínima.



Ejemplo guía

app/seguridad/csrf.php

```
<?php
function tokenCsrf(): string
{
    if (empty($_SESSION['csrf'])) {
        $_SESSION['csrf'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf'];
}

function validarCsrf(?string $enviado): bool
{
    return is_string($enviado)
        && !empty($_SESSION['csrf'])
        && hash_equals($_SESSION['csrf'], $enviado);
}
```

app/controladores/login.php

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/../config/conexion.php';
require_once __DIR__ . '/../seguridad/csrf.php';
session_start();

$error = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    if (!validarCsrf($_POST['csrf'] ?? null)) {
        http_response_code(419);
        exit('Solicitud no válida. Recargue el formulario.');
```

```
    }

    $correo = trim((string)($_POST['correo'] ?? ''));
    $clave = (string)($_POST['clave'] ?? '');

    if (!filter_var($correo, FILTER_VALIDATE_EMAIL) || strlen($clave) < 8) {
        $error = 'Correo o contraseña incorrectos.';
    } else {
        $pdo = Conexion::obtener();

        $st = $pdo->prepare(
            "SELECT id, nombre, clave_hash, rol, activo, bloqueado_hasta
            FROM usuarios WHERE correo = :correo LIMIT 1"
        );
        $st->execute([':correo' => $correo]);
        $u = $st->fetch();

        if ($u && $u['bloqueado_hasta'] !== null
            && strtotime($u['bloqueado_hasta']) > time()) {
            $error = 'Cuenta bloqueada temporalmente. Intente más tarde.';
        } elseif ($u && (int)$u['activo'] === 1
            && password_verify($clave, $u['clave_hash'])) {

            /* Migrar el hash si el costo del algoritmo subió */
            if (password_needs_rehash($u['clave_hash'], PASSWORD_DEFAULT)) {
                $nuevo = password_hash($clave, PASSWORD_DEFAULT);
                $pdo->prepare("UPDATE usuarios SET clave_hash = :h WHERE id = :id")
                    ->execute([':h' => $nuevo, ':id' => $u['id']]);
            }

            registrarIntento($pdo, $correo, true);
        }
    }
}
```

```

        abrirSesion($u); // se implementa el día 11
        header('Location: /dashboard.php');
        exit;

    } else {
        registrarIntento($pdo, $correo, false);
        $error = 'Correo o contraseña incorrectos.'; // mensaje genérico
    }
}
}
}

```

Y en el formulario, el token y el escape de la salida:

```

<form method="post" action="login.php" autocomplete="on">
  <input type="hidden" name="csrf" value="<?= htmlspecialchars(tokenCsrf(), ENT_QUOTES, 'UTF-8')
  ?>">
  <?php if ($error !== ''): ?>
    <p class="alerta-error" role="alert">
      <?= htmlspecialchars($error, ENT_QUOTES, 'UTF-8') ?>
    </p>
  <?php endif; ?>
  ...
</form>

```

Actividad

1. Crear la tabla `usuarios` (`id` , `nombre` , `correo` único, `clave_hash` de 255 caracteres, `rol` , `activo` , `bloqueado_hasta` , `creado_en`) y la tabla `intentos_acceso` .
2. Implementar `registro.php` con `password_hash()` y crear tres usuarios: un administrador, un vendedor y un consultor.
3. Implementar `login.php` completo con token CSRF, mensajes genéricos y bloqueo tras cinco intentos fallidos en quince minutos.
4. Verificar en phpMyAdmin que la columna `clave_hash` contiene cadenas que empiezan por `$2y$` y no la contraseña.
5. Prueba de XSS: registrar un usuario cuyo nombre sea `<script>alert(1)</script>` y demostrar que se muestra como texto y no se ejecuta.

Evidencia del día

Ingreso funcional, captura de la tabla `usuarios` con los hashes, captura del bloqueo tras el quinto intento y captura de la prueba de XSS neutralizada.

Criterio de evaluación

Cumple si no existe ninguna contraseña legible en la base de datos, el token CSRF se valida en cada envío y el mensaje de error es idéntico para correo inexistente y contraseña equivocada. No cumple si usa `md5()` , `sha1()` o compara contraseñas con `==` .

Objetivo

Sostener la identidad del usuario entre peticiones de forma segura, proteger todas las páginas del sistema y diferenciar lo que cada rol puede ver y hacer.

Contenidos

- Qué es una sesión: el servidor guarda los datos y el navegador solo lleva el identificador en una cookie. Ciclo de vida: `session_start()` , uso, destrucción.
- **Fijación de sesión** y su remedio: `session_regenerate_id(true)` inmediatamente después de autenticar y al cambiar de privilegios.
- Endurecer la cookie de sesión: `httponly` (inaccesible desde JavaScript), `samesite=Strict` , `secure` bajo HTTPS y `use_strict_mode` .
- Expiración por inactividad y expiración absoluta. Marca de tiempo de la última actividad en la propia sesión.
- Huella del cliente: verificar que el agente de usuario no cambie durante la sesión.
- Cierre de sesión correcto: vaciar el arreglo, destruir la sesión y caducar la cookie. No basta con `unset` .
- Guardián de acceso (`guardia.php`) incluido al inicio de toda página privada, y autorización por rol: administrador, vendedor y consultor.
- Diferencia entre *autenticación* (quién es) y *autorización* (qué puede hacer).

Ciclo de vida de la sesión



Autorización por rol

Acción	Administrador	Vendedor	Consultor
Ver tablero	Sí	Sí	Sí
Crear y editar productos	Sí	Sí	No
Eliminar registros	Sí	No	No
Gestionar usuarios	Sí	No	No

Ocultar un botón en el menú no es seguridad: la restricción se verifica también en el servidor.

Figura 11. Arriba, los cinco momentos de una sesión. Abajo, la matriz de permisos que el menú lateral del día 12 reflejará visualmente.

Ejemplo guía

app/seguridad/sesion.php

```
<?php
declare(strict_types=1);

const INACTIVIDAD_MAX = 1800; // 30 minutos
const SESSION_MAX     = 28800; // 8 horas

function iniciarSesionSegura(): void
{
    if (session_status() === PHP_SESSION_ACTIVE) return;

    ini_set('session.use_strict_mode', '1');
    session_set_cookie_params([
        'lifetime' => 0,
        'path'     => '/',
        'httponly' => true, // JavaScript no puede leerla
        'secure'   => !empty($_SERVER['HTTPS']),
        'samesite' => 'Strict',
    ]);
    session_name('ZDTL_SESS');
    session_start();
}

function abrirSesion(array $u): void
{
    iniciarSesionSegura();
    session_regenerate_id(true); // nuevo ID: evita fijación

    $_SESSION['usuario'] = [
        'id'     => (int)$u['id'],
        'nombre' => $u['nombre'],
        'rol'    => $u['rol'],
    ];
    $_SESSION['inicio']      = time();
    $_SESSION['ultima_actividad'] = time();
    $_SESSION['huella']      = hash('sha256', $_SERVER['HTTP_USER_AGENT'] ?? '');
}

function cerrarSesion(): void
{
    iniciarSesionSegura();
    $_SESSION = [];
    if (ini_get('session.use_cookies')) {
        $p = session_get_cookie_params();
        setcookie(session_name(), '', time() - 42000,
            $p['path'], $p['domain'], $p['secure'], $p['httponly']);
    }
    session_destroy();
}
```

app/seguridad/guardia.php — se incluye en la primera línea de cada página privada:

```
<?php
require_once __DIR__ . '/sesion.php';
iniciarSesionSegura();

/* 1. ¿Hay sesión? */
if (empty($_SESSION['usuario'])) {
    header('Location: /login.php?m=requiere_ingreso'); exit;
}

/* 2. ¿Es el mismo navegador? */
if ((($_SESSION['huella'] ?? '') !== hash('sha256', $_SERVER['HTTP_USER_AGENT'] ?? ''))) {
    cerrarSesion();
}
```

```

        header('Location: /login.php?m=sesion_invalida'); exit;
    }

    /* 3. Inactividad y duración máxima */
    $ahora = time();
    if ($ahora - $_SESSION['ultima_actividad'] > INACTIVIDAD_MAX
        || $ahora - $_SESSION['inicio'] > SESSION_MAX) {
        cerrarSesion();
        header('Location: /login.php?m=sesion_expirada'); exit;
    }
    $_SESSION['ultima_actividad'] = $ahora;

    /* 4. Autorización por rol */
    function exigirRol(string ...$roles): void
    {
        if (!in_array($_SESSION['usuario']['rol'], $roles, true)) {
            http_response_code(403);
            exit('403 - No tiene permiso para esta operación.');
```

Uso en una página restringida:

```

<?php
require_once __DIR__ . '/app/seguridad/guardia.php';
exigirRol('administrador'); // solo el administrador llega aquí
$usuario = $_SESSION['usuario'];
?>
<p>Bienvenido, <?= htmlspecialchars($usuario['nombre'], ENT_QUOTES, 'UTF-8') ?></p>
```

Actividad

1. Implementar `sesion.php`, `guardia.php` y `salir.php`, e incluir el guardián en todas las páginas privadas.
2. Mostrar en la barra superior el nombre y el rol del usuario autenticado, escapados con `htmlspecialchars`.
3. Prueba de acceso directo: cerrar sesión y escribir `dashboard.php` en la barra de direcciones. Debe redirigir al ingreso.
4. Prueba de rol: ingresar como consultor e intentar abrir `usuarios.php` directamente por URL. Debe responder 403.
5. Prueba de expiración: reducir `INACTIVIDAD_MAX` a 60 segundos, esperar y recargar. Documentar el resultado y devolver el valor a 1800.
6. Comprobar en las herramientas del navegador que la cookie de sesión tiene marcados `HttpOnly` y `SameSite`.

Evidencia del día

Capturas de las cuatro pruebas (acceso directo, 403 por rol, expiración e inspección de la cookie) y del identificador de sesión antes y después de autenticar, demostrando que cambió.

Criterio de evaluación

Cumple si ninguna página privada es alcanzable sin sesión, el ID se regenera al autenticar y el cierre destruye la sesión y caduca la cookie. No cumple si la protección consiste únicamente en ocultar enlaces del menú.

12^{DÍA}

Dashboard funcional con menú lateral izquierdo

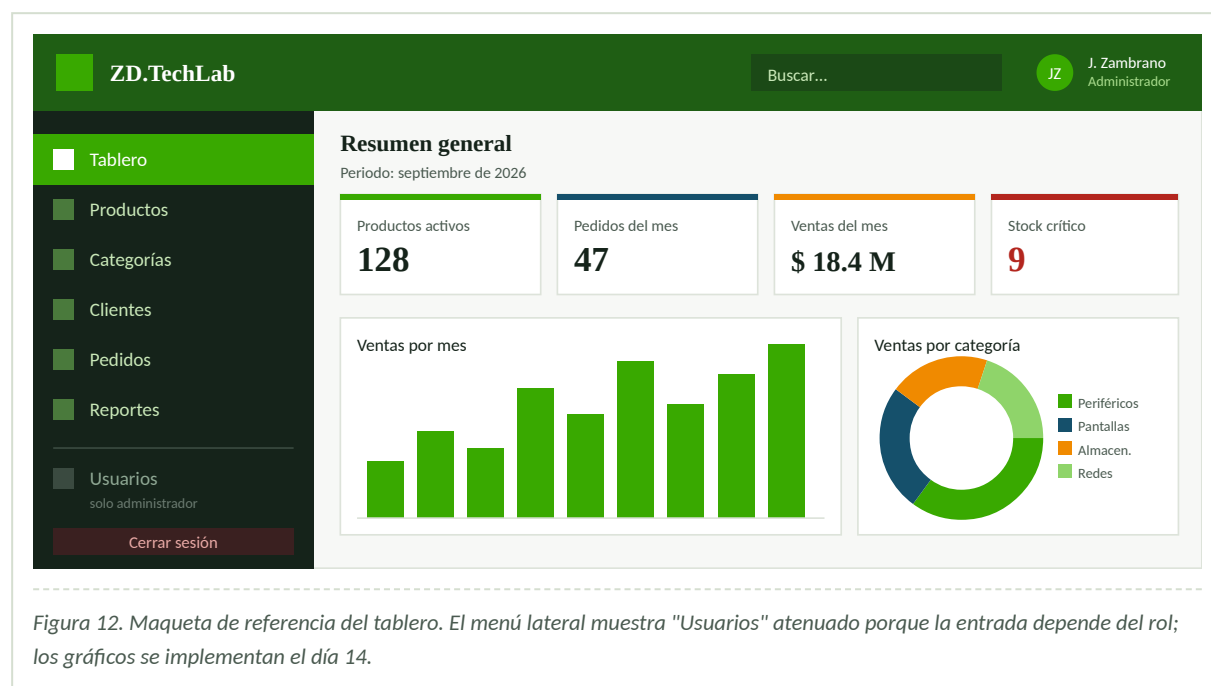
4 horas · Codificación

Objetivo

Convertir la maqueta del día 5 en un tablero real: una plantilla única que integra la sesión, muestra indicadores calculados desde la base de datos y ofrece un menú lateral que cambia según el rol del usuario.

Contenidos

- Plantillas parciales en PHP: `cabecera.php`, `menu.php` y `pie.php` incluidos con `require` para no repetir código en cada pantalla.
- Menú lateral construido desde un arreglo de configuración, con el rol requerido declarado en cada entrada. Se dibuja solo lo que el usuario puede usar, y además el servidor lo verifica.
- Marcar el elemento activo del menú comparando el nombre del archivo actual.
- Consultas de agregación para las tarjetas de indicadores: `COUNT`, `SUM`, `AVG`, `MAX` con `WHERE` por rango de fechas.
- Accesibilidad del menú: navegación por teclado, `aria-current="page"` en el enlace activo y `aria-expanded` en el botón que lo colapsa.
- Rutas relativas contra rutas absolutas y la constante `BASE_URL`.



Ejemplo guía

app/vistas/parciales/menu.php

```
<?php
$paginaActual = basename($_SERVER['PHP_SELF']);

$opciones = [
    ['archivo'=>'dashboard.php', 'texto'=>'Tablero', 'roles'=>
    ['administrador','vendedor','consultor']],
    ['archivo'=>'productos.php', 'texto'=>'Productos', 'roles'=>
    ['administrador','vendedor','consultor']],
    ['archivo'=>'categorias.php', 'texto'=>'Categorías', 'roles'=>['administrador','vendedor']],
    ['archivo'=>'clientes.php', 'texto'=>'Clientes', 'roles'=>['administrador','vendedor']],
    ['archivo'=>'pedidos.php', 'texto'=>'Pedidos', 'roles'=>['administrador','vendedor']],
    ['archivo'=>'reportes.php', 'texto'=>'Reportes', 'roles'=>['administrador','consultor']],
    ['archivo'=>'usuarios.php', 'texto'=>'Usuarios', 'roles'=>['administrador']],
];
?>

<aside class="panel__menu" id="menu-lateral">
    <nav aria-label="Menú principal">
        <ul class="menu">
            <?php foreach ($opciones as $op): ?>
                <?php if (!puede(...$op['roles'])) continue; ?>
                <?php $activo = ($paginaActual === $op['archivo']); ?>
                <li>
                    <a href="<?= BASE_URL . $op['archivo'] ?>"
                        class="menu__enlace<?= $activo ? ' menu__enlace--activo' : '' ?>"
                        <?= $activo ? 'aria-current="page"' : '' ?>
                        <?= htmlspecialchars($op['texto'], ENT_QUOTES, 'UTF-8') ?>
                    </a>
                </li>
            <?php endforeach; ?>
        </ul>
        <a href="<?= BASE_URL ?>salir.php" class="menu__salir">Cerrar sesión</a>
    </nav>
</aside>
```

Indicadores en `dashboard.php` :

```
<?php
require_once __DIR__ . '/app/seguridad/guardia.php';
require_once __DIR__ . '/app/config/conexion.php';
$pdo = Conexion::obtener();

$indicadores = $pdo->query("
    SELECT
        (SELECT COUNT(*) FROM productos WHERE activo = 1) AS productos_activos,
        (SELECT COUNT(*) FROM pedidos
            WHERE YEAR(fecha)=YEAR(CURDATE()) AND MONTH(fecha)=MONTH(CURDATE())) AS pedidos_mes,
        (SELECT COALESCE(SUM(total),0) FROM pedidos
            WHERE YEAR(fecha)=YEAR(CURDATE()) AND MONTH(fecha)=MONTH(CURDATE())) AS ventas_mes,
        (SELECT COUNT(*) FROM productos WHERE stock < 5 AND activo = 1) AS stock_critico
    ")>fetch();

require __DIR__ . '/app/vistas/parciales/cabecera.php';
require __DIR__ . '/app/vistas/parciales/menu.php';
?>

<main class="panel__contenido">
    <h1>Resumen general</h1>
    <section class="indicadores">
        <article class="tarjeta tarjeta--verde">
            <p class="tarjeta__rotulo">Productos activos</p>
            <p class="tarjeta__valor"><?= (int)$indicadores['productos_activos'] ?></p>
        </article>
        <article class="tarjeta tarjeta--naranja">
            <p class="tarjeta__rotulo">Ventas del mes</p>
```

```
<p class="tarjeta__valor">$ <?= number_format((float)$indicadores['ventas_mes'], 0, ',',  
'.') ?></p>  
</article>  
</section>  
</main>  
<?php require __DIR__ . '/app/vistas/parciales/pie.php'; ?>
```

Actividad

1. Extraer cabecera, menú y pie a parciales y reescribir todas las páginas para usarlos.
2. Construir el menú lateral desde el arreglo de configuración, con marcado del elemento activo.
3. Implementar las cuatro tarjetas de indicadores con datos reales de la base de datos.
4. Conectar el botón de colapso del menú (día 8) al tablero real y verificar el manejo de `aria-expanded`.
5. Comparar el menú visible como administrador, como vendedor y como consultor, y adjuntar las tres capturas.

Evidencia del día

Tablero navegable con indicadores reales y las tres capturas del menú por rol.

Criterio de evaluación

Cumple si el menú se genera desde configuración y no está copiado en cada archivo, los indicadores provienen de consultas reales y el elemento activo se marca correctamente. No cumple si los valores de las tarjetas están escritos a mano en el HTML.

13^{DÍA}

CRUD completo sobre la base de datos

4 horas · Codificación · Mayor carga del plan

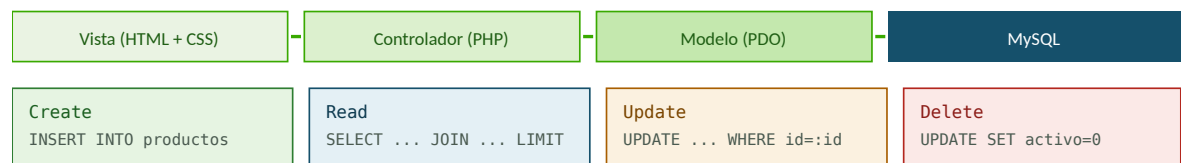
Objetivo

Construir las cuatro operaciones de persistencia para dos módulos completos, con validación en servidor, mensajes de resultado y protección contra doble envío.

Contenidos

- Correspondencia entre operaciones y SQL: crear → `INSERT`, leer → `SELECT`, actualizar → `UPDATE`, eliminar → `DELETE`.
- Separación en capas: modelo (consultas), controlador (decisiones) y vista (presentación). Es el patrón MVC en su forma más simple.
- Patrón `POST / Redirect / GET`: después de guardar se redirige, para que recargar la página no duplique el registro.
- Mensajes efímeros en sesión: guardar el aviso, mostrarlo una vez y borrarlo.
- **Borrado lógico** frente a borrado físico: en un sistema con historial de ventas casi nunca se elimina de verdad; se marca `activo = 0`. Manejo de `ON DELETE RESTRICT` cuando hay referencias.
- Confirmación antes de eliminar, con la acción real ejecutada por `POST`, nunca por un enlace `GET`.
- Transacciones con `beginTransaction`, `commit` y `rollBack` para el registro de un pedido y su detalle.
- Paginación con `LIMIT` y `OFFSET`, y ordenamiento seguro mediante lista blanca de columnas.

Las cuatro operaciones y sus capas



Patrón POST / Redirect / GET



Sin la redirección, pulsar F5 vuelve a enviar el formulario y se duplica el registro.

Producto	Categoría	Precio	Stock	Acciones
Teclado mecánico RGB	Periféricos	\$ 120.000	14	<button>Editar</button> <button>Eliminar</button>

Figura 13. El recorrido de un registro desde el formulario hasta la tabla. Los botones de la fila llevan el identificador en un atributo `data-id`, como se practicó el día 8.

Ejemplo guía

Modelo — `app/modelos/ProductoModelo.php`

```
<?php
declare(strict_types=1);

final class ProductoModelo
{
    public function __construct(private PDO $pdo) {}

    public function listar(string $busqueda = '', int $pagina = 1, int $porPagina = 10): array
    {
        $offset = ($pagina - 1) * $porPagina;
        $st = $this->pdo->prepare(
            "SELECT p.id, p.nombre, p.precio, p.stock, c.nombre AS categoria
            FROM productos p
            INNER JOIN categorias c ON c.id = p.categoria_id
            WHERE p.activo = 1 AND p.nombre LIKE :b
            ORDER BY p.nombre
            LIMIT :lim OFFSET :off"
        );
        $st->bindValue(':b', '%' . $busqueda . '%');
        $st->bindValue(':lim', $porPagina, PDO::PARAM_INT);
        $st->bindValue(':off', $offset, PDO::PARAM_INT);
        $st->execute();
        return $st->fetchAll();
    }

    public function crear(array $d): int
    {
        $st = $this->pdo->prepare(
            "INSERT INTO productos (nombre, categoria_id, precio, stock, activo, creado_en)
            VALUES (:nombre, :categoria, :precio, :stock, 1, NOW())"
        );
        $st->execute([
            'nombre' => $d['nombre'],
            'categoria' => $d['categoria_id'],
            'precio' => $d['precio'],
            'stock' => $d['stock'],
        ]);
        return (int)$this->pdo->lastInsertId();
    }

    public function actualizar(int $id, array $d): bool
    {
        $st = $this->pdo->prepare(
            "UPDATE productos
            SET nombre = :nombre, categoria_id = :categoria,
            precio = :precio, stock = :stock
            WHERE id = :id"
        );
        return $st->execute([
            'nombre' => $d['nombre'],
            'categoria' => $d['categoria_id'],
            'precio' => $d['precio'],
            'stock' => $d['stock'],
            'id' => $id,
        ]);
    }

    /* Borrado lógico: el historial de ventas debe seguir siendo legible */
    public function desactivar(int $id): bool
    {
        return $this->pdo
            ->prepare("UPDATE productos SET activo = 0 WHERE id = :id")
            ->execute(['id' => $id]);
    }
}
```

```
}
}
```

Controlador — validación en servidor, redirección y aviso:

```
<?php
require_once __DIR__ . '/app/seguridad/guardia.php';
exigirRol('administrador', 'vendedor');

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    if (!validarCsrf($_POST['csrf'] ?? null)) { http_response_code(419); exit; }

    $errores = [];
    $nombre = trim((string)$_POST['nombre'] ?? '');
    $precio = filter_input(INPUT_POST, 'precio', FILTER_VALIDATE_FLOAT);
    $stock = filter_input(INPUT_POST, 'stock', FILTER_VALIDATE_INT);
    $catId = filter_input(INPUT_POST, 'categoria_id', FILTER_VALIDATE_INT);

    if (mb_strlen($nombre) < 3) $errores[] = 'El nombre debe tener al menos 3
caracteres.';
    if ($precio === false || $precio <= 0) $errores[] = 'El precio debe ser mayor que cero.';
    if ($stock === false || $stock < 0) $errores[] = 'El stock no puede ser negativo.';
    if (!$catId) $errores[] = 'Seleccione una categoría.';

    if (!$errores) {
        $modelo = new ProductoModelo(Conexion::obtener());
        $id = (int)($_POST['id'] ?? 0);

        $id > 0
            ? $modelo->actualizar($id, compact('nombre','precio','stock') +
['categoria_id'=>$catId])
            : $modelo->crear(compact('nombre','precio','stock') + ['categoria_id'=>$catId]);

        $_SESSION['aviso'] = ['tipo'=>'exito', 'texto'=>'Producto guardado correctamente.'];
        header('Location: productos.php', true, 303); // POST / Redirect / GET
        exit;
    }
    $_SESSION['aviso'] = ['tipo'=>'error', 'texto'=>implode(' ', $errores)];
}
```

Transacción para el pedido y su detalle:

```
$pdo->beginTransaction();
try {
    $pdo->prepare("INSERT INTO pedidos (cliente_id, fecha, total) VALUES (:c, NOW(), :t)")
        ->execute([':c' => $clienteId, ':t' => $total]);
    $pedidoId = (int)$pdo->lastInsertId();

    $det = $pdo->prepare(
        "INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario)
        VALUES (:p, :pr, :c, :pu)"
    );
    foreach ($items as $it) {
        $det->execute([':p'=>$pedidoId, ':pr'=>$it['id'], ':c'=>$it['cant'],
':pu'=>$it['precio']]);
        $pdo->prepare("UPDATE productos SET stock = stock - :c WHERE id = :id")
            ->execute([':c'=>$it['cant'], ':id'=>$it['id']]);
    }
    $pdo->commit();
} catch (Throwable $e) {
    $pdo->rollBack(); // o todo, o nada
    error_log($e->getMessage());
    $_SESSION['aviso'] = ['tipo'=>'error', 'texto'=>'No fue posible registrar el pedido.'];
}
```

Actividad

1. Implementar el CRUD completo de **productos** con búsqueda, paginación de diez registros y ordenamiento por columna con lista blanca.
2. Implementar el CRUD completo de **clientes**, con validación de correo y documento único.
3. Implementar el registro de **pedidos** con detalle, dentro de una transacción que descuenta el stock.
4. Aplicar borrado lógico y demostrar que un producto desactivado desaparece del listado pero sigue apareciendo en los pedidos históricos.
5. Prueba de robustez: enviar el formulario con campos vacíos, con precio negativo y con texto en el campo numérico. Documentar los tres mensajes de error del servidor, con JavaScript desactivado en el navegador.

Evidencia del día

Dos CRUD funcionales, registro de pedidos con transacción y video de 90 segundos recorriendo crear, editar, buscar, paginar y desactivar. Capturas de la prueba con JavaScript desactivado.

Criterio de evaluación

Cumple si la validación funciona con JavaScript desactivado, existe redirección después de guardar y el borrado no rompe la integridad referencial. No cumple si eliminar se hace por un enlace **GET** o si recargar duplica registros.

14^{DÍA}

Vistas en la base de datos y gráficos con Chart.js

4 horas · Codificación

Objetivo

Trasladar la lógica de consolidación a la base de datos mediante vistas, exponer esos datos como JSON y representarlos con Chart.js dentro del tablero y de los reportes.

Contenidos

- Qué es una **vista**: una consulta guardada con nombre que se usa como si fuera una tabla. Ventajas: centraliza la regla de negocio, simplifica el PHP y permite otorgar permisos sin exponer las tablas base.
- Sintaxis `CREATE OR REPLACE VIEW`, y sus límites: una vista con agregación no es actualizable.
- Agregación con `GROUP BY`, `HAVING`, funciones de fecha y `LEFT JOIN` para no perder categorías sin ventas.
- Entregar datos al navegador: un archivo PHP que responde `Content-Type: application/json`, o incrustar el JSON en la página con `json_encode` y las banderas de escape adecuadas.
- Chart.js: tipos `bar`, `line`, `doughnut` y `pie`. Estructura de `data.labels` y `data.datasets`, opciones de escala, leyenda, `tooltip` y formato de moneda.
- Aplicar la paleta del día 2 a los gráficos, y garantizar que sigan siendo distinguibles sin color (patrones o etiquetas de valor).
- `responsive` y `maintainAspectRatio`; destruir la instancia antes de redibujar para evitar fugas de memoria.

De las tablas al gráfico

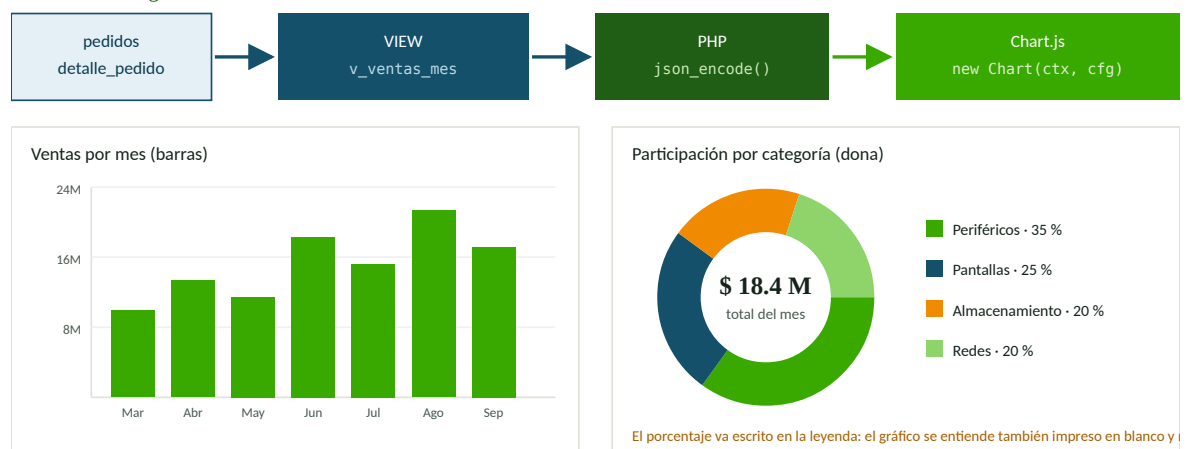


Figura 14. Los dos gráficos obligatorios del tablero. Ambos se alimentan de vistas, no de consultas escritas dentro de la página.

Ejemplo guía

sql/vistas.sql

```
-- Ventas consolidadas por mes
CREATE OR REPLACE VIEW v_ventas_mes AS
SELECT
    DATE_FORMAT(p.fecha, '%Y-%m') AS periodo,
    COUNT(DISTINCT p.id) AS cantidad_pedidos,
    SUM(d.cantidad) AS unidades,
    SUM(d.cantidad * d.precio_unitario) AS total_vendido
FROM pedidos p
INNER JOIN detalle_pedido d ON d.pedido_id = p.id
WHERE p.estado = 'confirmado'
GROUP BY DATE_FORMAT(p.fecha, '%Y-%m');

-- Participación por categoría (LEFT JOIN para no perder categorías sin ventas)
CREATE OR REPLACE VIEW v_ventas_categoria AS
SELECT
    c.id AS categoria_id,
    c.nombre AS categoria,
    COALESCE(SUM(d.cantidad), 0) AS unidades,
    COALESCE(SUM(d.cantidad * d.precio_unitario), 0) AS total_vendido
FROM categorias c
LEFT JOIN productos pr ON pr.categoria_id = c.id
LEFT JOIN detalle_pedido d ON d.producto_id = pr.id
LEFT JOIN pedidos p ON p.id = d.pedido_id AND p.estado = 'confirmado'
GROUP BY c.id, c.nombre;

-- Productos bajo el punto de reposición
CREATE OR REPLACE VIEW v_stock_critico AS
SELECT pr.id, pr.nombre, c.nombre AS categoria, pr.stock, pr.stock_minimo
FROM productos pr
INNER JOIN categorias c ON c.id = pr.categoria_id
WHERE pr.activo = 1 AND pr.stock <= pr.stock_minimo;
```

api/graficos.php — protegido por el mismo guardián de sesión:

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/../app/seguridad/guardia.php';
require_once __DIR__ . '/../app/config/conexion.php';

header('Content-Type: application/json; charset=utf-8');
$pdo = Conexion::obtener();

$meses = $pdo->query(
    "SELECT periodo, total_vendido FROM v_ventas_mes
    ORDER BY periodo DESC LIMIT 12"
)->fetchAll();
$meses = array_reverse($meses);

$categorias = $pdo->query(
    "SELECT categoria, total_vendido FROM v_ventas_categoria
    WHERE total_vendido > 0 ORDER BY total_vendido DESC"
)->fetchAll();

echo json_encode([
    'ventasMes' => [
        'etiquetas' => array_column($meses, 'periodo'),
        'valores' => array_map('floatval', array_column($meses, 'total_vendido')),
    ],
    'categorias' => [
        'etiquetas' => array_column($categorias, 'categoria'),
        'valores' => array_map('floatval', array_column($categorias, 'total_vendido')),
    ]
]);
```

```
],  
], JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR);
```

js/graficos.js

```
const PALETA = ["#39A900", "#15506B", "#F08A00", "#8FD46A", "#B0209E", "#00A0C6"];  
const moneda = new Intl.NumberFormat("es-CO",  
  { style:"currency", currency:"COP", maximumFractionDigits:0 });  
  
let graficoVentas, graficoCategorias;  
  
async function dibujarGraficos(){  
  const datos = await fetch("api/graficos.php", { credentials:"same-origin" })  
    .then(r => r.json());  
  
  graficoVentas?.destroy();  
  graficoVentas = new Chart(document.getElementById("g-ventas"), {  
    type: "bar",  
    data: {  
      labels: datos.ventasMes.etiquetas,  
      datasets: [{  
        label: "Ventas del mes",  
        data: datos.ventasMes.valores,  
        backgroundColor: PALETA[0],  
        borderRadius: 2  
      }]  
    },  
    options: {  
      responsive: true,  
      maintainAspectRatio: false,  
      plugins: {  
        legend: { display:false },  
        tooltip: { callbacks: { label: c => moneda.format(c.parsed.y) } }  
      },  
      scales: {  
        y: { beginAtZero:true,  
          ticks:{ callback: v => (v/1_000_000) + " M" } }  
      }  
    }  
  });  
  
  graficoCategorias?.destroy();  
  graficoCategorias = new Chart(document.getElementById("g-categorias"), {  
    type: "doughnut",  
    data: {  
      labels: datos.categorias.etiquetas,  
      datasets: [{ data: datos.categorias.valores, backgroundColor: PALETA }]  
    },  
    options: {  
      responsive: true,  
      maintainAspectRatio: false,  
      cutout: "62%",  
      plugins: {  
        legend: { position:"right" },  
        tooltip: { callbacks: { label: c => `${c.label}: ${moneda.format(c.parsed)} ` } }  
      }  
    }  
  });  
}  
  
document.addEventListener("DOMContentLoaded", dibujarGraficos);
```

Actividad

1. Crear en `sql/vistas.sql` un mínimo de cuatro vistas: ventas por mes, ventas por categoría, stock crítico y clientes con mayor compra acumulada.
2. Implementar `api/graficos.php` y verificar que, sin sesión activa, no devuelve datos.
3. Incorporar al tablero al menos tres gráficos de tipos distintos, alimentados por las vistas.
4. Agregar un filtro de rango de fechas que redibuje los gráficos sin recargar la página.
5. Verificar en el inspector de red que la respuesta JSON es válida y que ninguna consulta SQL aparece en el código del navegador.

Evidencia del día

`sql/vistas.sql`, capturas del resultado de cada vista ejecutada en phpMyAdmin, tablero con los tres gráficos y captura del JSON en el inspector de red.

Criterio de evaluación

Cumple si los gráficos consumen vistas y no consultas incrustadas en la página, los tipos son pertinentes al dato (serie temporal en barras o líneas, participación en dona) y los valores se muestran formateados en pesos. No cumple si los datos de los gráficos están escritos a mano en el JavaScript.

15
DÍA

Reportes con identidad institucional y sustentación final

4 horas · Codificación + evaluación

Objetivo

Emitir reportes imprimibles y exportables que incluyan el logo y los datos del software, con tablas, totales y gráficos, y sustentar la aplicación completa ante el instructor.

Contenidos

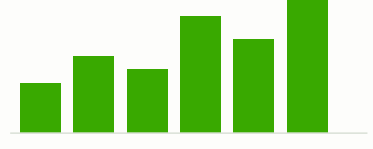
- Anatomía de un reporte formal: encabezado con logo, nombre del software, título del reporte, filtros aplicados, fecha y hora de generación y usuario que lo generó; cuerpo con la tabla de datos; totales; y pie con paginación y una nota de confidencialidad.
- Reportes basados en vistas y en consultas parametrizadas por rango de fechas, categoría o estado.
- Impresión desde el navegador con `@media print`: ocultar el menú, fijar `@page`, forzar colores con `print-color-adjust` y evitar cortes con `break-inside: avoid`.
- Exportación a PDF con Dompdf instalado por Composer, y por qué conviene generar el mismo HTML para pantalla y para PDF.
- Insertar el gráfico en el PDF: `canvas.toDataURL()` convierte el gráfico de Chart.js en una imagen incrustable.
- Exportación a CSV con `fputcsv` y el marcador BOM para que Excel respete los acentos.
- Logo en dos versiones: SVG para pantalla, PNG en base64 para el PDF.



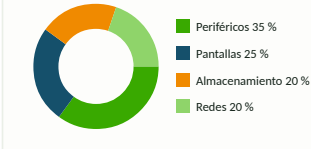
Reporte de ventas por categoría

Filtros: 01/01/2026 al 13/09/2026 - Estado: confirmado - Todas las categorías

Evolución mensual



Participación



Categoría	Unidades	Pedidos	Total vendido	%
Periféricos	412	96	\$ 6.440.000	35,0
Pantallas	58	44	\$ 4.600.000	25,0
Almacenamiento	137	61	\$ 3.680.000	20,0
Redes	93	38	\$ 3.680.000	20,0
Total general	700	239	\$ 18.400.000	100

Figura 15. Estructura obligatoria del reporte: encabezado con logo y datos del software, filtros aplicados, gráficos, tabla con fila de totales y pie con paginación.

Ejemplo guía

Encabezado reutilizable — `app/vistas/reportes/encabezado.php`

```
<?php
/* Datos del software, centralizados en un solo lugar */
const APP = [
    'nombre' => 'ZD.TechLab',
    'lema' => 'Sistema de gestión de tienda tecnológica',
    'version' => '1.0',
    'nit' => '900.123.456-7',
    'ciudad' => 'Bogotá D.C.',
    'correo' => 'soporte@zdttechlab.co',
    'logo' => __DIR__ . '/../assets/img/logo.png',
];

function logoBase64(): string
{
    return 'data:image/png;base64,' . base64_encode(file_get_contents(APP['logo']));
}
?>
<header class="reporte__encabezado">
    " class="reporte__logo">
    <div class="reporte__marca">
        <h1><?= APP['nombre'] ?></h1>
        <p><?= APP['lema'] ?> · v<?= APP['version'] ?></p>
        <p>NIT <?= APP['nit'] ?> · <?= APP['ciudad'] ?> · <?= APP['correo'] ?></p>
    </div>
    <div class="reporte__meta">
        <p>Generado: <?= date('d/m/Y H:i') ?></p>
        <p>Usuario: <?= htmlspecialchars($_SESSION['usuario']['nombre'], ENT_QUOTES, 'UTF-8') ?>
            (<?= htmlspecialchars($_SESSION['usuario']['rol'], ENT_QUOTES, 'UTF-8') ?>)</p>
        <p>Reporte N.º <?= $numeroReporte ?></p>
    </div>
</header>
```

Estilos de impresión — `css/reporte.css`

```
@page { size: A4 portrait; margin: 14mm 12mm 18mm; }

@media print{
    .panel__menu, .panel__barra, .no-imprimir { display:none !important; }
    body { background:#fff; font-size:10.5pt; }
    .reporte__encabezado{
        border-bottom:2.5px solid var(--color-accion);
        -webkit-print-color-adjust: exact;
        print-color-adjust: exact;
    }
    table { width:100%; border-collapse:collapse; }
    thead { display: table-header-group; } /* repite el encabezado en cada página */
    tr, img, .grafico { break-inside: avoid; }
    .reporte__pie{ position:fixed; bottom:0; width:100%; font-size:8pt; }
}
```

Exportación a PDF con el gráfico incrustado:

```
// En el navegador: el canvas se convierte en imagen y viaja al servidor
document.querySelector("#btn-pdf").addEventListener("click", async () => {
    const cuerpo = new URLSearchParams({
        csrf: document.querySelector("#csrf").value,
        desde: document.querySelector("#desde").value,
        hasta: document.querySelector("#hasta").value,
        grafico1: document.querySelector("#g-ventas").toDataURL("image/png", 1.0),
        grafico2: document.querySelector("#g-categorias").toDataURL("image/png", 1.0)
    });
```

```
const r = await fetch("reportes/ventas-pdf.php",
    { method:"POST", body:cuerpo, credentials:"same-origin" });
window.open(URL.createObjectURL(await r.blob()));
});
```

```
<?php
/* reportes/ventas-pdf.php */
require_once __DIR__ . '/../app/seguridad/guardia.php';
exigirRol('administrador', 'consultor');
if (!validarCsrf($_POST['csrf'] ?? null)) { http_response_code(419); exit; }

require_once __DIR__ . '/../vendor/autoload.php'; // composer require dompdf/dompdf

$pdo = Conexion::obtener();
$stmt = $pdo->prepare(
    "SELECT categoria, unidades, total_vendido
    FROM v_ventas_categoria
    WHERE total_vendido > 0 ORDER BY total_vendido DESC"
);
$stmt->execute();
$filas = $stmt->fetchAll();
$total = array_sum(array_column($filas, 'total_vendido'));

ob_start();
require __DIR__ . '/../app/vistas/reportes/ventas.php'; // el mismo HTML de pantalla
$html = ob_get_clean();

$dmpdf = new \Dompdf\Dompdf(['isRemoteEnabled' => true]);
$dmpdf->loadHtml($html, 'UTF-8');
$dmpdf->setPaper('A4', 'portrait');
$dmpdf->render();
$dmpdf->stream('reporte-ventas-' . date('Ymd-Hi') . '.pdf', ['Attachment' => false]);
```

Exportación a CSV legible en Excel:

```
<?php
header('Content-Type: text/csv; charset=utf-8');
header('Content-Disposition: attachment; filename="ventas-' . date('Ymd') . '.csv"');

$salida = fopen('php://output', 'w');
fwrite($salida, "\xEF\xBB\xBF"); // BOM: acentos correctos en Excel
fputcsv($salida, ['Categoría', 'Unidades', 'Total vendido'], ';');
foreach ($filas as $f) {
    fputcsv($salida, [$f['categoria'], $f['unidades'], $f['total_vendido']], ';');
}
fclose($salida);
```

Actividad

1. Centralizar los datos del software en **APP** y construir el encabezado reutilizable de reportes.
2. Construir tres reportes, cada uno alimentado por una vista y filtrable por rango de fechas: ventas por categoría, inventario con stock crítico, y pedidos por cliente.
3. Incluir en al menos dos de ellos un gráfico de Chart.js y una fila de totales.
4. Habilitar la impresión con **@media print** y la exportación a PDF y a CSV.
5. Imprimir un reporte en papel o a PDF y verificar: el logo aparece, el menú no, el encabezado de la tabla se repite en la segunda página y ninguna fila queda partida.
6. Preparar la sustentación: presentación de máximo diez minutos con recorrido de la aplicación.

Evidencia final del plan • entrega en Teams el 28/09/2026 a las 5:00 p. m.

- Repositorio con quince commits fechados, uno por día como mínimo.
- Aplicación funcionando, con `README.md` que explique instalación, credenciales de prueba y estructura de carpetas.
- `sql/estructura.sql` , `sql/datos.sql` y `sql/vistas.sql` .
- Tres reportes exportados en PDF.
- Bitácora completa de los quince días.
- Sustentación presencial ante el instructor.

Criterio de evaluación

Cumple si el reporte impreso conserva logo, identidad y totales, y si el mismo documento se obtiene en pantalla y en PDF sin discrepancias. No cumple si el PDF sale sin logo, con el menú lateral incrustado o con las tablas cortadas entre páginas.

4. Rúbrica de evaluación final

La calificación se emite después de la sustentación. Cada criterio se valora sobre el porcentaje indicado. El plan se aprueba con un mínimo del 70 % y con todos los criterios marcados como indispensables en estado "cumple".

#	Criterio	Peso	Indicador de logro
1	Fundamento de color y coherencia visual	8 %	Justifica la paleta con el esquema de armonía; todos los contrastes alcanzan WCAG AA.
2	HTML semántico y accesibilidad	7 %	Páginas validan en W3C sin errores; formularios con <code>label</code> y foco visible.
3	CSS, maquetación y responsive	10 %	Grid y Flexbox correctamente aplicados; sin desbordamiento horizontal en 360 px.
4	JavaScript y validación en cliente	8 %	Delegación de eventos, sin <code>onclick</code> en línea, mensajes junto al campo.
5	Inicio de sesión seguro (<i>indispensable</i>)	13 %	<code>password_hash</code> / <code>password_verify</code> , CSRF validado, mensajes genéricos, bloqueo por intentos.
6	Manejo de sesiones (<i>indispensable</i>)	12 %	Regeneración de ID, cookie <code>HttpOnly</code> / <code>SameSite</code> , expiración, guardián en todas las páginas, roles verificados en servidor.
7	Dashboard con menú lateral	8 %	Menú generado por configuración según rol, indicadores desde consultas reales, elemento activo marcado.
8	CRUD y persistencia (<i>indispensable</i>)	13 %	Dos módulos completos con sentencias preparadas, validación en servidor, POST/Redirect/GET y transacción en pedidos.
9	Consultas, vistas y gráficos	10 %	Mínimo cuatro vistas en uso; tres gráficos de Chart.js alimentados desde ellas.
10	Reportes con identidad del software	8 %	Logo, datos del software, filtros, totales, paginación; salida correcta en PDF y CSV.
11	Proceso: commits diarios y bitácora	3 %	Quince commits fechados y bitácora completa.
12	Sustentación y dominio del código (<i>indispensable</i>)	10 %	Explica cualquier fragmento solicitado y realiza en vivo las modificaciones pedidas.

Condición de cierre. Un aprendiz que obtenga un porcentaje aprobatorio pero no supere alguno de los cuatro criterios indispensables (5, 6, 8 y 12) no aprueba el plan. La seguridad del acceso, la persistencia de datos y el dominio del propio código no son negociables en un desarrollador de software.

5. Recursos de apoyo

- **Documentación de referencia:** MDN Web Docs (HTML, CSS, JavaScript), manual oficial de PHP, documentación de Chart.js y de Dompdf, guías WCAG 2.1 en español.
- **Herramientas:** XAMPP o Laragon, Visual Studio Code con extensiones PHP Intelephense y Live Server, Git y GitHub, phpMyAdmin o MySQL Workbench, Figma para el prototipo, WebAIM Contrast Checker, validador del W3C.
- **Acompañamiento:** asesoría del instructor en horario de ambiente de formación; revisión de avance obligatoria al cierre del día 6 y del día 11.

6. Compromiso y firmas

El aprendiz declara conocer el alcance de este plan, las fechas de entrega, los criterios de evaluación y las condiciones de sustentación. El instructor se compromete a brindar el acompañamiento y la realimentación descritos. Cualquier prórroga debe solicitarse por escrito, a través de Microsoft Teams, a más tardar el miércoles 23 de septiembre de 2026; después de esa fecha no se conceden ampliaciones.

Firma del aprendiz
Nombre y documento

Ing. José Andrés Zambrano Díaz
Instructor · ADSI

Visto bueno
Coordinación académica

Centro de Diseño y Metrología · Regional Distrito Capital · Servicio Nacional de Aprendizaje
— SENA
Elaborado por Ing. José Andrés Zambrano Díaz · Programa de Análisis y Desarrollo de Software